TECNOLÓGICO DE ESTUDIOS SUPERIORES DE ECATEPEC

DIVISIÓN DE INGENIERÍA EN SISTEMAS COMPUTACIONALES

PROGRAMACIÓN DE BASE DE DATOS

TAREA 1: CADENA DE CONEXIÓN A BASES DE DATOS CON GO

PROFESORA:

ALEJANDRA GONZALEZ SÁNCHEZ

NOMBRE:

MONTERRUBIO LÓPEZ LUIS GAEL

GRUPO: 5701

FECHA: 21-10-26

1. Introducción

Para que un programa pueda guardar o leer datos, primero tiene que conectarse a la base de datos. Para eso se usa la **cadena de conexión**, un texto que le dice al programa dónde está el servidor, a qué base de datos entrar y con qué usuario.

En esta tarea el caso es el proyecto *IoT Analytics Dashboard* de NexoDev Solutions, que guarda lecturas de sensores en PostgreSQL. Si la cadena está mal, el sistema no puede guardar ni consultar nada.

En este documento explico qué partes tiene una cadena de conexión y lo pruebo en la práctica: levanté PostgreSQL con Docker, hice un programa en Go que se conecta y subí todo a GitHub. También agregué MySQL para comparar cómo cambia la cadena entre dos bases de datos.

2. Objetivos

Objetivo general: conectar un programa en Go a PostgreSQL y MySQL, entendiendo cada parte de la cadena de conexión.

Objetivos específicos:

1. Investigar las partes de una cadena de conexión.
2. Crear un entorno de pruebas con Docker.
3. Hacer un programa en Go que se conecte a las bases de datos.
4. Documentar el proceso, incluyendo los errores y cómo los resolví.
5. Subir el código y la documentación a GitHub.

3. ¿Qué es una cadena de conexión?

Es un texto con los datos que necesita el programa para conectarse a la base de datos. El programa no se conecta solo: le pasa la cadena a un **driver** (un paquete especial para cada base de datos), y el driver hace la conexión.

Una forma fácil de verla es como una dirección completa: dice en qué equipo está la base de datos, por qué "puerta" entrar, a qué base ir y con qué usuario y contraseña.

3.1 Partes de la cadena

PARTE	PARA QUÉ SIRVE	VALOR QUE USÉ
Host	En qué equipo está el servidor	localhost (mi propia computadora)
Puerto	Por dónde se entra al servidor	5432 en PostgreSQL, 3306 en MySQL
Base de datos	A qué base de datos entrar	iot_analytics
Usuario	Con qué cuenta se entra	iot_user
Contraseña	La clave de esa cuenta	iot_pass
Parámetros adicionales	Opciones extra, como el cifrado	sslmode=disable, parseTime=true

Los parámetros adicionales van al final, después de un `?`. Si son varios, se separan con `&`.

3.2 Cadena en PostgreSQL

```
postgres://usuario:contraseña@host:puerto/base_de_datos?parametros
postgres://iot_user:iot_pass@localhost:5432/iot_analytics?sslmode=disable
```

Usé `sslmode=disable` porque es una prueba en mi computadora y no hace falta cifrar la conexión. En un servidor real se usaría `sslmode=require`.

3.3 Cadena en MySQL

```
usuario:contraseña@tcp(host:puerto)/base_de_datos?parametros
iot_user:iot_pass@tcp(localhost:3306)/iot_analytics?parseTime=true
```

`parseTime=true` sirve para que las fechas de la base de datos lleguen a Go como fechas y no como texto. Es útil porque las lecturas de los sensores tienen fecha y hora.

3.4 Diferencias

Las dos cadenas llevan los mismos datos; lo que cambia es cómo se escriben:

	POSTGRESQL	MYSQL
Cómo empieza	postgres://	Directo con el usuario
Host y puerto	@localhost:5432	@tcp(localhost:3306)
Puerto por defecto	5432	3306
Driver en Go	pgx	go-sql-driver/mysql

4. Herramientas utilizadas

HERRAMIENTA	PARA QUÉ LA USÉ
Docker Desktop	Correr PostgreSQL y MySQL sin instalarlos directamente en Windows
WSL 2	Lo necesita Docker para funcionar en Windows
PostgreSQL 16 y MySQL 8	Las dos bases de datos que conecté
Go	El lenguaje en el que hice el programa
Drivers pgx y go-sql-driver/mysql	Permiten que Go se comunique con cada base de datos
Git y GitHub	Guardar y entregar el código

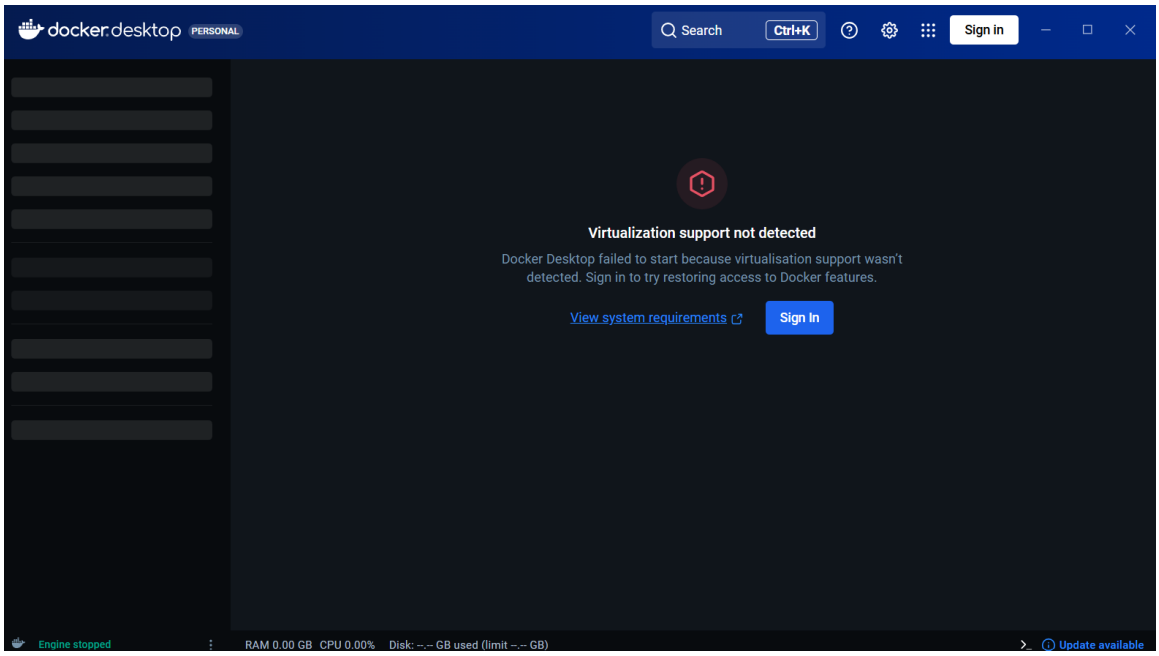
Go tiene un paquete llamado `database/sql` que funciona igual para cualquier base de datos. Por eso pude usar el mismo código para las dos; solo cambié el driver y la cadena.

5. Preparación del entorno

Antes de crear los contenedores fue necesario resolver un error de Docker Desktop. Se documenta porque es un problema común en equipos con Windows y forma parte del proceso real de configuración.

5.1 Problema detectado

Al abrir Docker Desktop apareció el mensaje **"Virtualization support not detected"** y el motor quedó en estado **Engine stopped**. Docker necesita virtualización por hardware para ejecutar la máquina virtual ligera donde corren los contenedores.



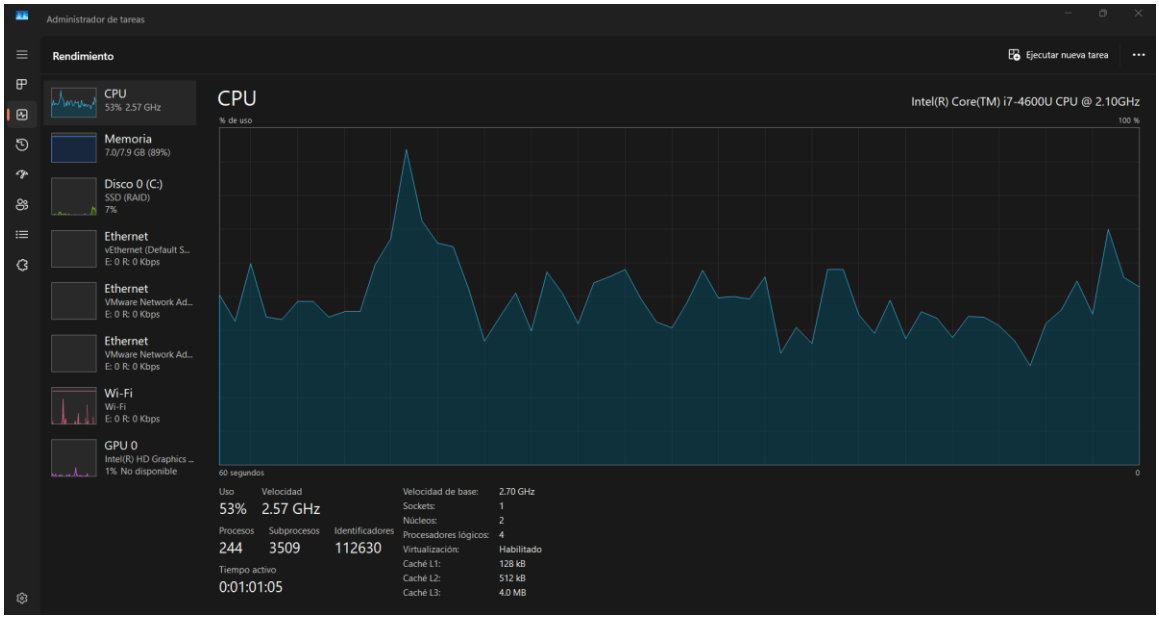
Docker Desktop no inicia: "Virtualization support not detected" y motor en estado Engine stopped.

5.2 Diagnóstico

Se revisó si el procesador tenía la virtualización activada:

1. Abrir el Administrador de tareas con `Ctrl + Shift + Esc`.
2. Entrar a la pestaña **Rendimiento** y seleccionar **CPU**.
3. Revisar el campo **Virtualización**.

El equipo (procesador Intel Core i7-4600U) mostró **Virtualización: Habilitado**. Esto indicó que el problema no estaba en la BIOS, sino en Windows: faltaba el componente **Plataforma de máquina virtual** que usa WSL 2.



Administrador de tareas, pestaña Rendimiento > CPU: Virtualización "Habilitado" en el procesador Intel Core i7-4600U.

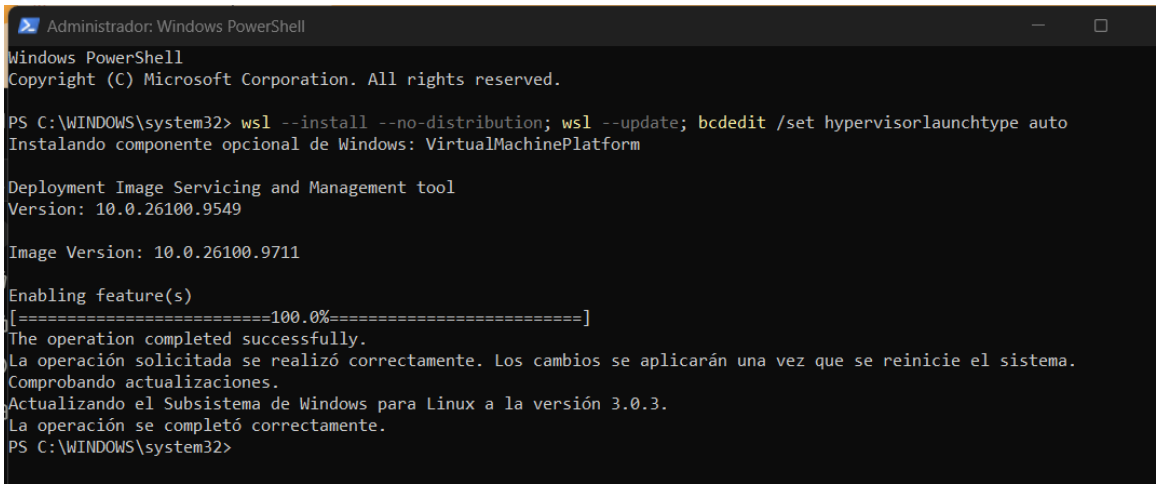
5.3 Solución

En PowerShell, ejecutado como administrador, se corrió el siguiente comando:

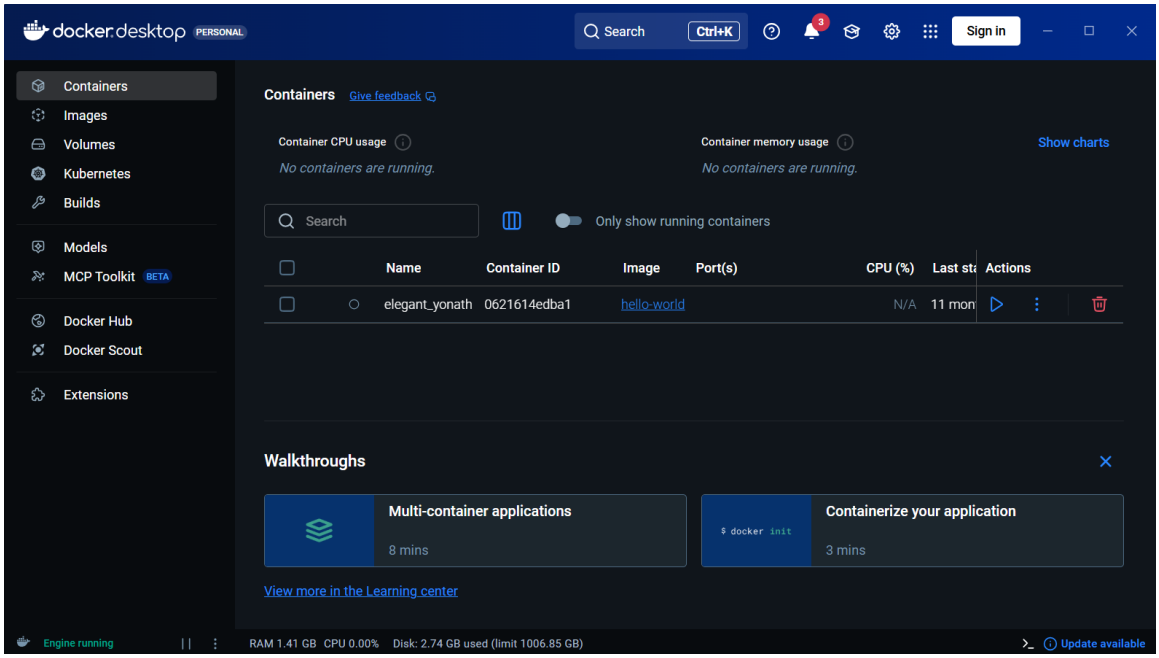
```
wsl --install --no-distribution; wsl --update; bcdedit /set hypervisorlaunchtype auto
```

PARTE DEL COMANDO	QUÉ HACE
wsl --install --no-distribution	Instala WSL y activa la Plataforma de máquina virtual, sin descargar una distribución de Linux
wsl --update	Actualiza WSL a la versión más reciente
bcdedit /set hypervisorlaunchtype auto	Indica a Windows que inicie el hipervisor al arrancar el sistema

La terminal confirmó la instalación de *VirtualMachinePlatform* y la actualización de WSL a la versión 3.0.3. Después de reiniciar el equipo, Docker Desktop mostró **Engine running**.



PowerShell como administrador: se habilita VirtualMachinePlatform y se actualiza WSL a la versión 3.0.3.



Después de reiniciar, Docker Desktop muestra "Engine running" y queda listo para crear contenedores.

También se cerraron aplicaciones que no se usaban, ya que el equipo cuenta con 8 GB de RAM y la memoria estaba cerca del 90 %. Docker y los dos servidores de base de datos requieren memoria disponible para funcionar con fluidez.

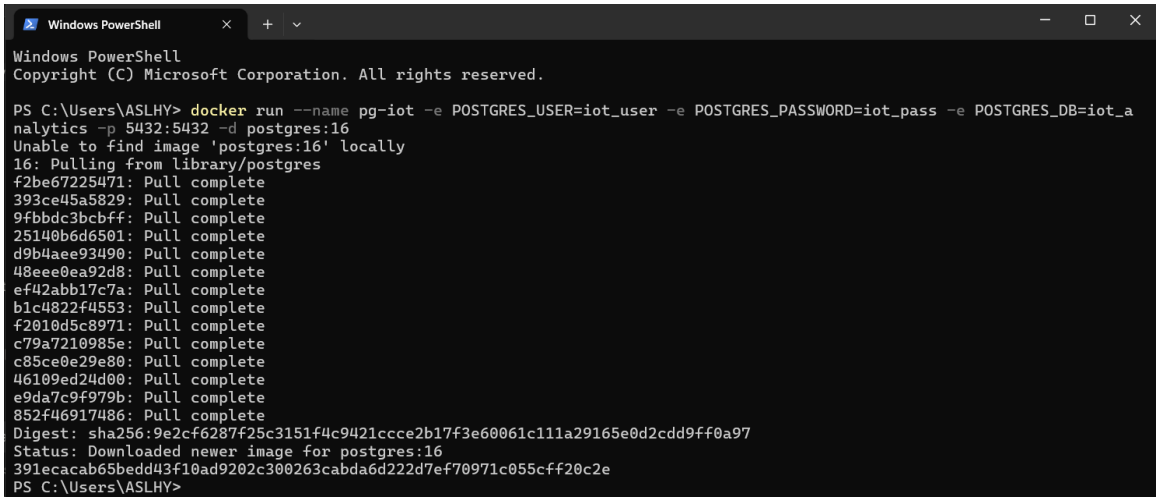
6. Despliegue de las bases de datos en Docker

Cada SGBD se ejecuta en su propio contenedor. Así no se instala nada directamente en Windows, y el entorno se puede borrar y volver a crear en segundos.

6.1 Contenedor de PostgreSQL

```
docker run --name pg-iot -e POSTGRES_USER=iot_user -e POSTGRES_PASSWORD=iot_pass -e POSTGRES_DB=iot_analytics -p 5432:5432 -d postgres:16
```

Como la imagen `postgres:16` no existía en el equipo, Docker la descargó de Docker Hub por capas (*Pull complete*) y después creó el contenedor. La última línea es el identificador del contenedor creado.

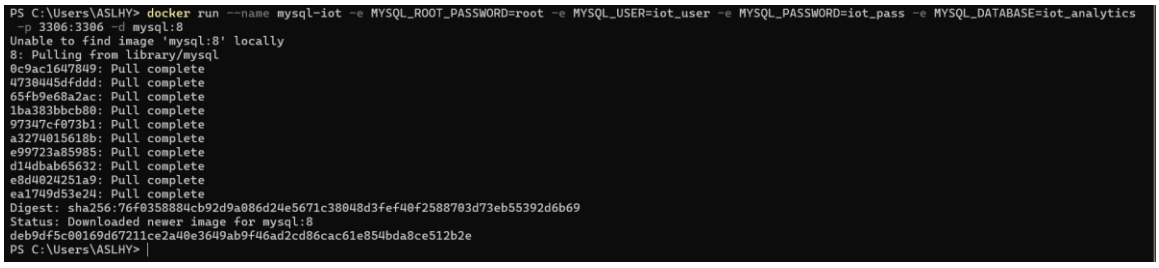


Descarga de la imagen postgres:16 y creación del contenedor pg-iot.

6.2 Contenedor de MySQL

```
docker run --name mysql-iot -e MYSQL_ROOT_PASSWORD=root -e MYSQL_USER=iot_user -e MYSQL_PASSWORD=iot_pass -e MYSQL_DATABASE=iot_analytics -p 3306:3306 -d mysql:8
```

El proceso fue el mismo: Docker descargó la imagen `mysql:8` y creó el contenedor `mysql-iot`.



Descarga de la imagen mysql:8 y creación del contenedor mysql-iot.

6.3 Explicación de las opciones

OPCIÓN	SIGNIFICADO
docker run	Crea e inicia un contenedor a partir de una imagen
--name pg-iot	Nombre del contenedor, para identificarlo fácilmente
-e VARIABLE=valor	Variable de entorno que la imagen usa para configurarse al primer arranque
POSTGRES_USER / MYSQL_USER	Crea el usuario de la aplicación
POSTGRES_PASSWORD / MYSQL_PASSWORD	Define la contraseña de ese usuario
POSTGRES_DB / MYSQL_DATABASE	Crea la base de datos iot_analytics
MYSQL_ROOT_PASSWORD	Contraseña del administrador de MySQL; la imagen la exige
-p 5432:5432	Publica el puerto: puerto_en_Windows:puerto_en_el_contenedor
-d	Ejecuta el contenedor en segundo plano
postgres:16 / mysql:8	Imagen oficial y versión que se descarga de Docker Hub

6.4 Relación con la cadena de conexión

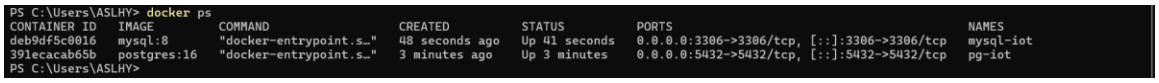
Los valores usados al crear los contenedores son exactamente los que después van en la cadena de conexión:

DATO EN DOCKER	COMPONENTE DE LA CADENA
-p 5432:5432 y -p 3306:3306	Host localhost y puertos 5432 / 3306
POSTGRES_USER, MYSQL_USER	Usuario iot_user
POSTGRES_PASSWORD, MYSQL_PASSWORD	Contraseña iot_pass
POSTGRES_DB, MYSQL_DATABASE	Base de datos iot_analytics

6.5 Verificación

`docker ps`

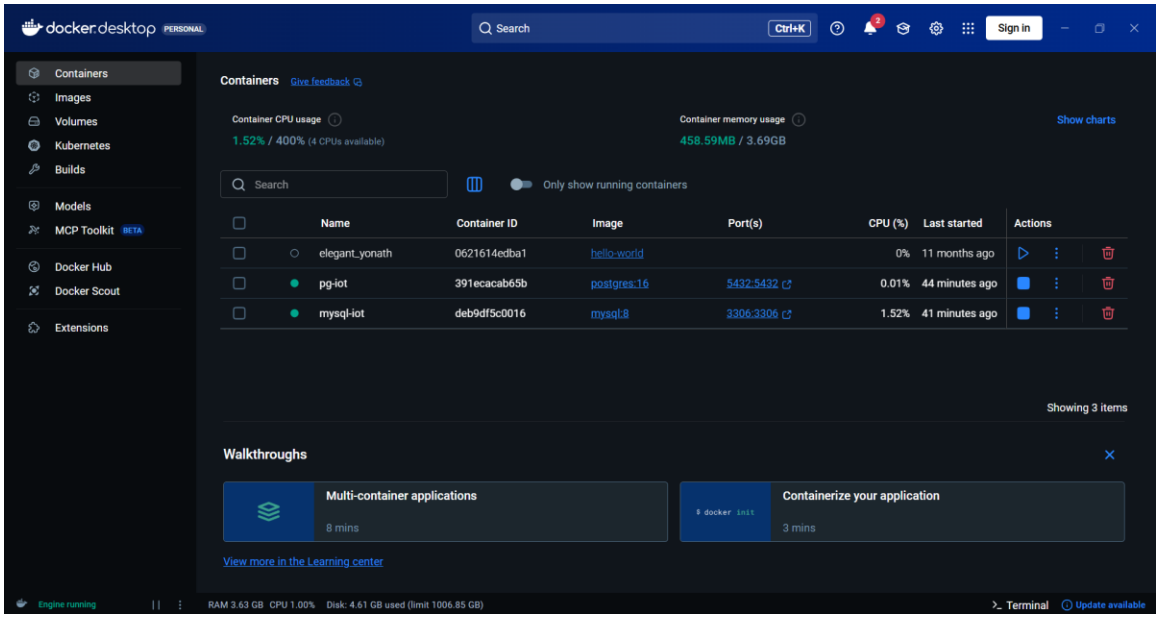
Este comando lista los contenedores en ejecución. Ambos deben aparecer con estado **Up** y con sus puertos publicados.



docker ps muestra los contenedores mysql-iot y pg-iot en estado Up.

La columna **PORTS** confirma la publicación de puertos: `0.0.0.0:5432->5432/tcp` para PostgreSQL y `0.0.0.0:3306->3306/tcp` para MySQL. Esto significa que cualquier programa en Windows puede llegar a cada servidor usando `localhost` y su puerto, que son justo el host y el puerto que irán en las cadenas de conexión.

Lo mismo se puede comprobar en Docker Desktop: los dos contenedores aparecen con el punto verde (encendidos), su imagen y sus puertos. El contenedor `hello-world` es una prueba anterior de Docker y no forma parte de esta tarea.



Contenedores pg-iot y mysql-iot en ejecución en Docker Desktop.

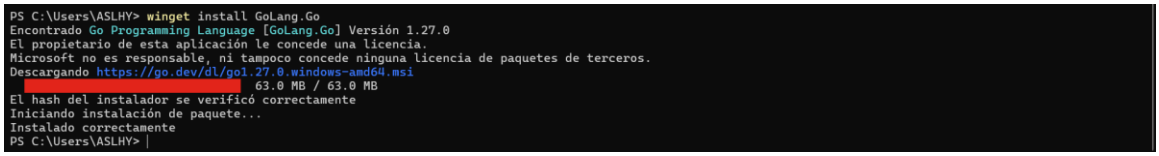
7. Script de conexión en Go

7.1 Instalación de Go

Al ejecutar `go version` por primera vez, PowerShell respondió que el término `go` no se reconocía: el lenguaje no estaba instalado. Se instaló con **winget**, el gestor de paquetes de Windows:

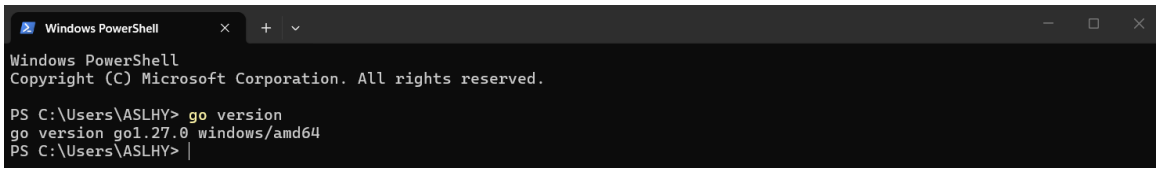
```
winget install GoLang.Go
```

winget descargó el instalador oficial de go.dev (versión 1.27.0), verificó su hash para confirmar que el archivo no estaba alterado y lo instaló.



Instalación de Go 1.27.0 mediante winget.

Después se cerró y se volvió a abrir PowerShell. Este paso es necesario porque la terminal carga la variable de entorno `PATH` solo al iniciar; sin reabrirla, no encuentra el ejecutable `go`.



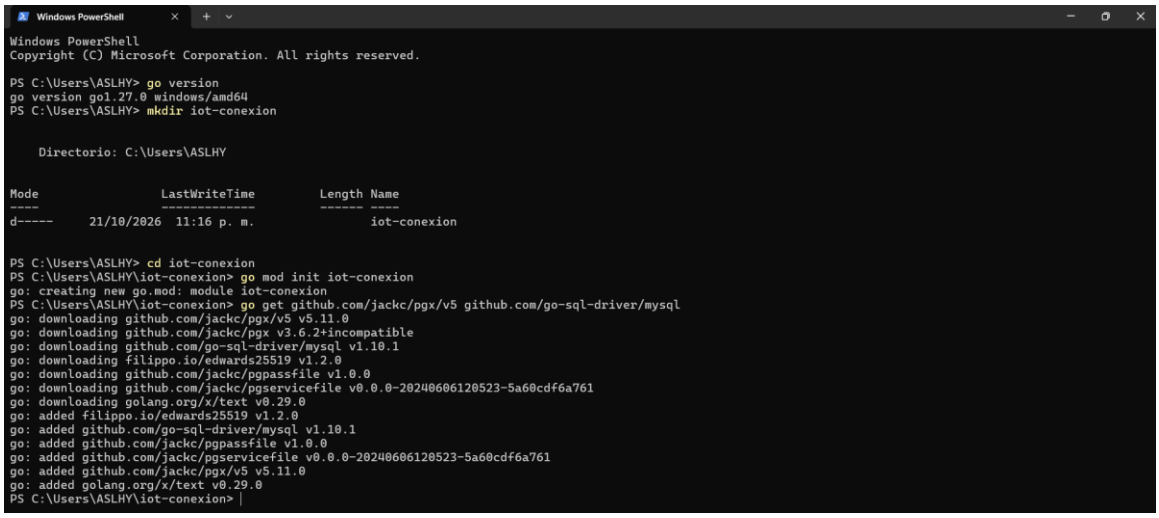
`go version` confirma la instalación: go1.27.0 windows/amd64.

7.2 Creación del proyecto

```
mkdir iot-conexion
cd iot-conexion
go mod init iot-conexion
go get github.com/jackc/pgx/v5 github.com/go-sql-driver/mysql
```

COMANDO	QUÉ HACE
go mod init iot-conexion	Crea el archivo go.mod, que identifica el proyecto y registra sus dependencias
go get ...	Descarga los drivers de PostgreSQL y MySQL y los agrega a go.mod

La salida muestra que go mod init creó el archivo go.mod y que go get descargó los drivers pgx/v5 (v5.11.0) y go-sql-driver/mysql (v1.10.1). También descargó sus dependencias, como pgpassfile y golang.org/x/text, que Go registra automáticamente.



Creación de la carpeta iot-conexion, inicialización del módulo y descarga de los drivers de PostgreSQL y MySQL.

7.3 Código completo (main.go)

```
package main

import (
	"database/sql"
	"fmt"
	"log"

	_ "github.com/go-sql-driver/mysql"
	_ "github.com/jackc/pgx/v5/stdlib"
)

func conectar(driver, dsn string) {
	db, err := sql.Open(driver, dsn)
	if err != nil {
		log.Fatal(err)
	}
	defer db.Close()

	if err := db.Ping(); err != nil {
		log.Fatalf("%s: %v", driver, err)
	}

	var version string
	db.QueryRow("SELECT version()").Scan(&version)
	fmt.Printf("✓ %s conectado: %s\n", driver, version)
```

```
}

func main() {
	// PostgreSQL: protocolo://usuario:contraseña@host:puerto/bd?parámetros
	conectar("pgx", "postgres://iot_user:iot_pass@localhost:5432/iot_analytics?sslmode=disable")

	// MySQL: usuario:contraseña@protocolo(host:puerto)/bd?parámetros
	conectar("mysql", "iot_user:iot_pass@tcp(localhost:3306)/iot_analytics?parseTime=true")
}
```

El archivo se creó con el Bloc de notas desde PowerShell (notepad main.go) dentro de la carpeta del proyecto.



Archivo main.go guardado con el script de conexión.

7.4 Explicación paso a paso

1. **Importaciones.** database/sql es la interfaz estándar de Go para bases de datos. Los drivers se importan con un guion bajo (_): no se usan por su nombre, pero al cargarse se registran como "pgx" y "mysql" para que sql.Open los reconozca.
2. **Función conectar.** Recibe el nombre del driver y la cadena de conexión. Se escribió una sola función para ambas bases de datos, lo que demuestra que solo cambian esos dos datos.
3. **sql.Open(driver, dsn).** Prepara el objeto de conexión y valida el formato de la cadena. Todavía no se comunica con el servidor.
4. **defer db.Close().** Garantiza que la conexión se cierre al terminar la función, aunque ocurra un error.
5. **db.Ping().** Aquí ocurre la conexión real: se abre el canal TCP, el servidor valida las credenciales y se selecciona la base de datos. Si algo de la cadena es incorrecto, el error aparece en este paso.
6. **SELECT version().** Consulta de prueba que funciona en ambos SGBD. Comprueba que, además de conectarse, la aplicación puede ejecutar instrucciones SQL.
7. **Función main.** Llama a conectar dos veces, una con cada cadena de conexión.

7.5 Ejecución

```
go run main.go
```

Problema encontrado: falta de entradas en go.sum. La primera ejecución se detuvo con el mensaje missing go.sum entry for module providing package github.com/jackc/puddle/v2. El archivo go.sum guarda las huellas (*checksums*) de cada dependencia para verificar que no fueron alteradas. Como pgx usa internamente otro paquete (puddle, para administrar el *pool* de conexiones) que no quedó registrado al ejecutar go get, Go se negó a compilar.

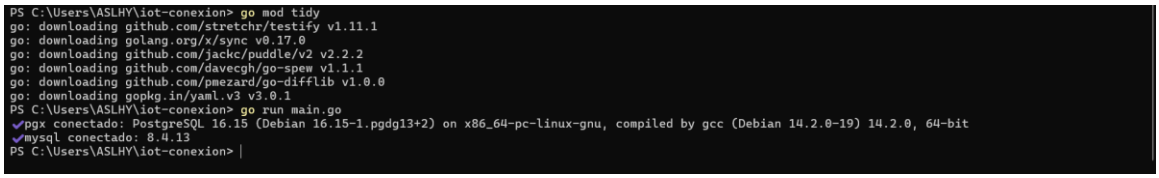


Error por dependencia sin registrar en go.sum.

Solución: ejecutar go mod tidy, que revisa las importaciones del código, descarga las dependencias faltantes y actualiza go.mod y go.sum:

```
go mod tidy
go run main.go
```

go mod tidy descargó puddle/v2 y las demás dependencias faltantes. Con eso, go run main.go compiló y ejecutó el script sin errores.



Salida de go mod tidy y conexión exitosa a PostgreSQL 16.15 y MySQL 8.4.13.

8. Pruebas y resultados

El script se conectó correctamente a las dos bases de datos y ejecutó una consulta en cada una (Figura 14).

SGBD	DRIVER	CADENA DE CONEXIÓN	RESULTADO	VERSIÓN REPORTADA POR EL SERVIDOR
PostgreSQL	pgx	postgres://iot_user:iot_pass@localhost:5432/iot_analytics?sslmode=disable	Conectado	PostgreSQL 16.15 (Debian), 64-bit
MySQL	mysql	iot_user:iot_pass@tcp(localhost:3306)/iot_analytics?parseTime=true	Conectado	8.4.13

8.1 Interpretación

- **La conexión de red funcionó.** El host localhost y los puertos 5432 y 3306 llegaron a los contenedores gracias a la publicación de puertos de Docker.
- **La autenticación fue válida.** db.Ping() no devolvió error, así que el usuario iot_user y su contraseña fueron aceptados por ambos servidores.
- **La base de datos existe y es accesible.** Ambos servidores abrieron iot_analytics, creada automáticamente por las variables de entorno de Docker.

- **Se pueden ejecutar consultas.** SELECT version() devolvió la versión de cada servidor. PostgreSQL entrega una descripción completa (versión, sistema operativo y compilador), mientras que MySQL solo devuelve el número de versión.
- **El mismo código sirvió para ambos SGBD.** Solo cambiaron el nombre del driver y la cadena de conexión, lo que confirma la ventaja de usar database/sql.

8.2 Problemas encontrados y soluciones

PROBLEMA	CAUSA	SOLUCIÓN
Docker Desktop: "Virtualization support not detected"	Faltaba la Plataforma de máquina virtual de Windows para WSL 2	wsl --install --no-distribution, wsl --update, bcdedit y reinicio (Sección 5)
go no se reconoce como comando	Go no estaba instalado	winget install GoLang.Go y reabrir PowerShell (Sección 7.1)
expected 'package', found 'EOF'	main.go estaba vacío: el código no se había guardado	Guardar el archivo con Ctrl + S
missing go.sum entry	Una dependencia interna de pgx no estaba registrada	go mod tidy (Sección 7.5)

9. Buenas prácticas

En esta tarea puse la contraseña directo en el código para que el ejemplo fuera más claro, pero en un proyecto real no se hace así. Lo recomendable es:

- **No escribir contraseñas en el código.** Se guardan en variables de entorno, por ejemplo os.Getenv("DATABASE_URL").
- **No subir contraseñas a GitHub.** Cualquiera que vea el repositorio podría usarlas.
- **Cifrar la conexión.** En producción se usa sslmode=require en lugar de disable.
- **Usar un usuario con permisos limitados**, como hice con iot_user, en lugar del administrador.

10. Repositorio Git

El código se publicó en GitHub para su revisión:

Repositorio: github.com/luisgael-ml/iot-conexion

10.1 Contenido del repositorio

ARCHIVO	DESCRIPCIÓN
Documentacion_ Tarea1	Documentación de la tarea
main.go	Script de conexión a PostgreSQL y MySQL
go.mod	Nombre del módulo y lista de dependencias (drivers)
go.sum	Huellas de verificación de cada dependencia

10.2 Comandos utilizados

```
git config --global user.name "Luis Gael Monterrubio López"
git config --global user.email "monterrubiolopezluisgael@gmail.com"
git init
git add .
git commit -m "Conexión a PostgreSQL y MySQL con Go"
git branch -M main
```

```
git remote add origin https://github.com/luisgael-ml/iot-conexion.git
git push -u origin main
```

COMANDO	QUÉ HACE
git config --global user.name / user.email	Registra el autor que aparecerá en cada *commit*
git init	Convierte la carpeta en un repositorio local (crea la carpeta oculta .git)
git add .	Prepara todos los archivos del proyecto para el siguiente *commit*
git commit -m "..."	Guarda una versión del proyecto con un mensaje que describe el cambio
git branch -M main	Nombra main a la rama principal
git remote add origin <url>	Vincula el repositorio local con el creado en GitHub
git push -u origin main	Sube los *commits* a GitHub y deja la rama local enlazada con la remota

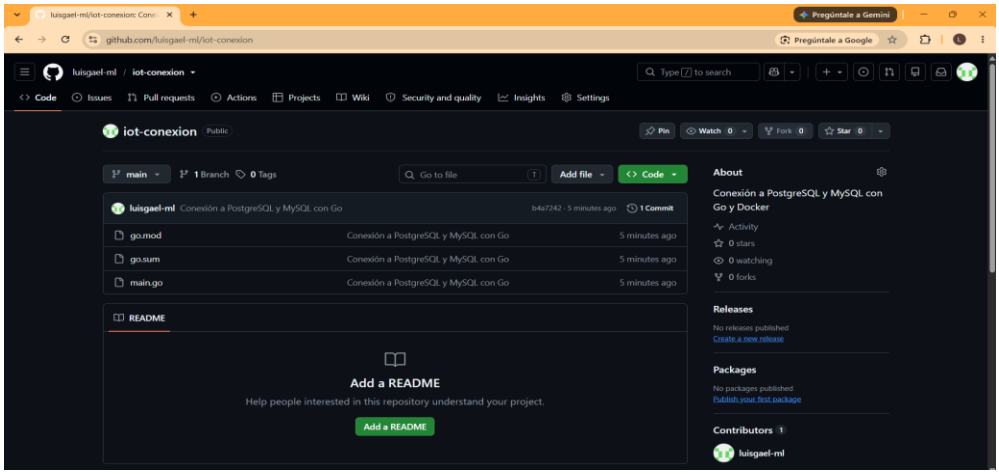
Al ejecutar `git push`, el Administrador de credenciales de Git abrió el navegador para iniciar sesión en GitHub y autorizar el acceso. Después se subieron los 5 objetos del *commit* `b4a7242` y se creó la rama `main` en el repositorio remoto.

Los avisos `LF will be replaced by CRLF` no son errores: Git solo informa que adapta los saltos de línea al formato de Windows.

```
PS C:\Users\ASLHY\iot-conexion> git config --global user.name "Luis Gael Monterrubio López"
PS C:\Users\ASLHY\iot-conexion> git config --global user.email "monterrubiolopezluisgael@gmail.com"
PS C:\Users\ASLHY\iot-conexion> git init
Initialized empty Git repository in C:/Users/ASLHY/iot-conexion/.git/
PS C:\Users\ASLHY\iot-conexion> git add .
warning: in the working copy of 'go.mod', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'go.sum', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\ASLHY\iot-conexion> git commit -m "Conexión a PostgreSQL y MySQL con Go"
[main (root-commit) b4a7242] Conexión a PostgreSQL y MySQL con Go
 3 files changed, 81 insertions(+)
 create mode 100644 go.mod
 create mode 100644 go.sum
 create mode 100644 main.go
PS C:\Users\ASLHY\iot-conexion> git branch -M main
PS C:\Users\ASLHY\iot-conexion> git remote add origin https://github.com/luisgael-ml/iot-conexion.git
PS C:\Users\ASLHY\iot-conexion> git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 2.21 KiB | 753.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/luisgael-ml/iot-conexion.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\ASLHY\iot-conexion>
```

Inicialización del repositorio, commit y envío exitoso a GitHub.

En GitHub se ven los tres archivos, el *commit* `b4a7242` con su mensaje y el autor `luisgael-ml`.



Repositorio público luisgael-ml/iot-conexion con main.go, go.mod y go.sum.

11. Conclusiones

Con esta práctica logré conectar un programa en Go a dos bases de datos, PostgreSQL y MySQL, que funcionaban dentro de Docker.

Lo más importante que aprendí es que la cadena de conexión es como una dirección completa: dice en qué equipo está la base de datos, por qué puerto entrar, a qué base de datos ir y con qué usuario y contraseña. Si uno solo de esos datos está mal, la conexión falla. Por eso los valores tienen que coincidir con los que se configuraron al crear los contenedores.

También vi que PostgreSQL y MySQL piden la misma información, aunque la escriben de forma diferente. Aun así, en Go pude usar el mismo código para las dos; solo cambié el driver y la cadena.

Durante el proceso tuve varios errores: Docker no iniciaba, Go no estaba instalado, el archivo no se había guardado y faltaban dependencias. Leer cada mensaje con calma me ayudó a entender qué pasaba y a resolverlo. Creo que eso también es parte de lo que se aprende al trabajar en un proyecto real.

Por último, entendí que en un sistema real no se deben escribir las contraseñas dentro del código y que la conexión debe ir cifrada para proteger la información.

Referencias

PostgreSQL Global Development Group. *Connection Strings* (libpq). postgresql.org/docs/current/libpq-connect.html

Go Authors. *Package database/sql*. pkg.go.dev/database/sql

Jackc. *pgx – PostgreSQL driver and toolkit for Go*. github.com/jackc/pgx

Go-SQL-Driver. *MySQL driver for Go – DSN (Data Source Name)*. github.com/go-sql-driver/mysql

Docker Inc. *Docker Official Image: postgres*. hub.docker.com/_/postgres

Docker Inc. *Docker Official Image: mysql*. hub.docker.com/_/mysql

Microsoft. *Instalación de WSL*. learn.microsoft.com/es-es/windows/wsl/install